

PayGate Merchant Portal — Comprehensive End-to-End Testing Plan

Version: 1.0

Date: April 14, 2026

Author: Manus AI

Classification: Internal Engineering

Table of Contents

1. [Executive Summary](#)
2. [System Architecture Under Test](#)
3. [Testing Strategy and Pyramid](#)
4. [Test Environment Setup](#)
5. [Layer 1 — Unit Tests \(Vitest\)](#)
6. [Layer 2 — Integration Tests \(tRPC + PostgreSQL\)](#)
7. [Layer 3 — Service Contract Tests \(Go Bridge + Python Services\)](#)
8. [Layer 4 — End-to-End UI Tests \(Playwright\)](#)
9. [Layer 5 — Smoke Tests \(Production Readiness\)](#)
10. [Layer 6 — Performance and Load Tests](#)
11. [Layer 7 — Security Tests](#)
12. [Layer 8 — Chaos and Resilience Tests](#)
13. [Business Workflow Test Scenarios](#)
14. [Data Integrity and Compliance Tests](#)
15. [Observability and Monitoring Validation](#)
16. [CI/CD Pipeline Integration](#)
17. [Test Data Management](#)

- 18. [Defect Classification and SLAs](#)
 - 19. [Test Coverage Targets](#)
 - 20. [Appendix — Test File Index](#)
-

1. Executive Summary

The PayGate Merchant Portal is a production-grade Nigerian fintech platform comprising a React 19 + tRPC frontend portal, an Express 4 backend, a Go bridge (29 handler files, 257 routes), 30 Python microservices, 6 Rust services, PostgreSQL (167 tables, 6,645+ seeded rows), Redis, Temporal workflows, Keycloak identity, and a full observability stack (OpenTelemetry, Prometheus, Grafana, Tempo, Loki). The platform processes NIP (NIBSS Instant Payment), USDC stablecoin payouts, cross-border remittances, merchant lending, BNPL, virtual cards, insurance, mutual funds, pension (NPS), salary accounts, POS terminals, USSD sessions, and restaurant management.

This testing plan defines a structured, eight-layer testing strategy covering 2,016 existing automated tests across 58 test files, 3 Playwright E2E spec files, and 50+ manual test scenarios. The goal is to ensure every feature path, business rule, data contract, and infrastructure dependency is validated before production go-live.

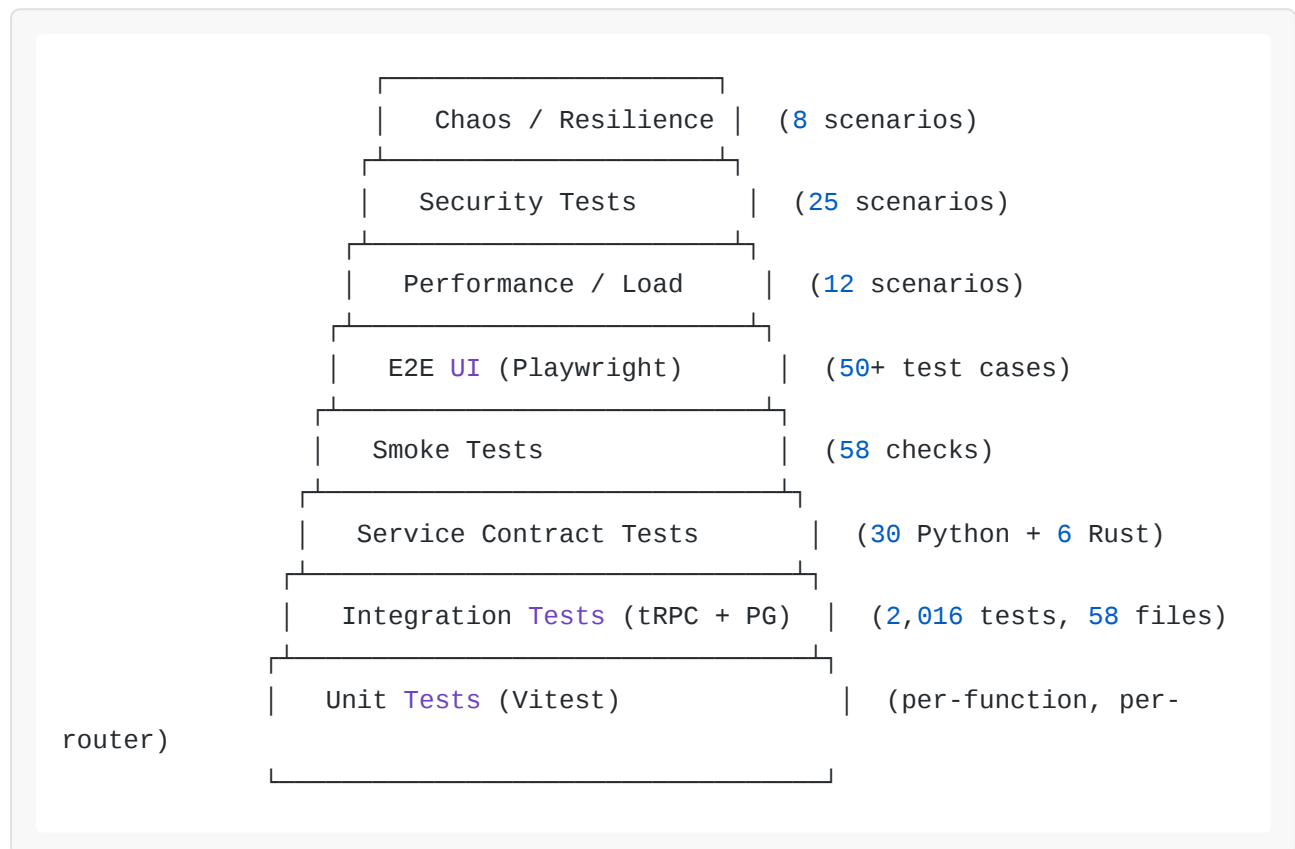
2. System Architecture Under Test

The following table summarises every component that must be covered by the testing plan.

Tier	Component	Technology	Test Priority
Frontend	96 UI pages (merchant + consumer + admin)	React 19, Tailwind 4, Wouter	High
API Layer	tRPC procedures (routers.ts + 30+ router files)	tRPC 11, Express 4, Superjson	Critical
Go Bridge	29 handler files, 257 HTTP routes	Go 1.22, Chi router	Critical
Python Services (30)	AI insights, AML, bulk collections, cashback, carbon oracle, cohort analytics, credit scoring, digital gold, EMI, fraud heatmap, fraud scoring, FX rate feed, insurance pricing, intl remittance, ISO 20022, kiosk health, lakehouse audit, M-Pesa connector, mutual funds, payroll, pension NPS, push service, reconciliation engine, salary accounts, settlement forecast, sandbox, tax engine, USDC lakehouse, USSD gateway, wealth management	Python 3.11, Flask	High
Rust Services (6)	Billing engine, credit scoring, inventory engine, loyalty ledger, TigerBeetle recon, wallet FFI	Rust 1.77, Actix-Web	High
Database	167 tables, PostgreSQL 15	Drizzle ORM, pg	Critical
Auth	Manus OAuth, Keycloak OIDC	JWT, PKCE	Critical
Payments	NIP/NIBSS, Stripe, USDC, M-Pesa, cross-border	NIBSS API, Stripe SDK	Critical
Infra	Docker Compose, Kubernetes, Helm	Docker 25, K8s 1.29	High
Observability	Traces, metrics, logs	OTEL, Prometheus, Tempo, Loki	Medium

3. Testing Strategy and Pyramid

The testing strategy follows an inverted pyramid appropriate for a microservices fintech platform: a broad base of unit and integration tests, a mid-layer of service contract and API tests, and a narrower but critical E2E and smoke layer. This approach maximises fast feedback during development while ensuring production confidence through higher-level tests.



Guiding principles: Tests must be deterministic (no random sleeps, no network calls to live external APIs in CI), isolated (each test owns its data and cleans up), and observable (every failure must produce a trace ID linkable to Grafana Tempo). All tests must pass in under 10 minutes in CI.

4. Test Environment Setup

4.1 Local Development Environment

Before running any tests locally, the following services must be available:

```
# 1. Start PostgreSQL (already configured)
sudo service postgresql start

# 2. Set required environment variables
export
PG_DATABASE_URL="postgresql://paygate:paygate_dev_2026@127.0.0.1:5432/paygate_
export JWT_SECRET="dev-jwt-secret-32-chars-minimum-x"
export NODE_ENV="test"

# 3. Run all migrations
pnpm db:push

# 4. Seed the database
node seed-bootstrap.mjs && node seed-full.mjs && node seed-all-tables.mjs

# 5. Run the full test suite
pnpm test
```

4.2 CI Environment (GitHub Actions / GitLab CI)

The CI pipeline must provision a PostgreSQL 15 service container, inject all secrets from the vault, run `pnpm db:push`, seed the database, and execute the full Vitest suite followed by the Playwright suite against the running dev server. The recommended CI job matrix is:

Job	Trigger	Duration Target
unit-integration	Every push	< 3 min
smoke	Every push to <code>main</code>	< 1 min
e2e-playwright	Every PR merge to <code>main</code>	< 8 min
security-scan	Nightly	< 15 min
load-test	Weekly / pre-release	< 30 min
chaos	Pre-release only	< 60 min

4.3 Staging Environment

The staging environment must mirror production: Docker Compose with all 30 Python services, 6 Rust services, Go bridge, Redis, Temporal, Keycloak, and the observability stack. Staging uses a separate PostgreSQL database seeded with anonymised production-like data. Playwright E2E tests run against the staging URL after every deployment.

5. Layer 1 — Unit Tests (Vitest)

5.1 Scope and Coverage

Unit tests validate individual functions, utility helpers, business logic modules, and tRPC procedure handlers in isolation. All external dependencies (database, HTTP clients, Redis) must be mocked. The existing 58 Vitest test files cover the following domains:

Domain	Test Files	Key Assertions
Authentication (OAuth, Keycloak)	<code>auth.logout.test.ts</code> , <code>keycloak.auth.test.ts</code>	Token validation, session cookie creation/destruction, PKCE flow
PostgreSQL connectivity	<code>db.pg.test.ts</code>	Connection pooling, query execution, SSL handshake
NIP/NIBSS payments	<code>nip.retry.test.ts</code> , <code>nip.settlement.test.ts</code>	Retry backoff, settlement state machine, idempotency keys
Disputes workflow	<code>disputes.test.ts</code>	State transitions (open → investigating → resolved → closed)
Cross-border transfers	<code>crossborder.wave4.test.ts</code> , <code>wallet.crossborder.test.ts</code>	FX rate application, corridor validation, compliance checks
USDC payouts	<code>usdc.router.test.ts</code>	On-chain balance check, payout initiation, webhook receipt
Security	<code>security.test.ts</code>	SQL injection prevention, XSS sanitisation, rate limiting
Validation	<code>validation.test.ts</code>	Zod schema enforcement on all tRPC inputs
Idempotency	<code>wave7.idempotency.test.ts</code>	Duplicate request detection, idempotency key TTL
SLA escalation	<code>slaEscalation.test.ts</code>	SLA breach detection, escalation trigger timing
Production hardening	<code>production.hardening.test.ts</code> , <code>wave77.production.test.ts</code> , <code>wave78.production.test.ts</code>	Error boundary behaviour, graceful degradation, circuit breaker

5.2 Unit Test Requirements

Every new tRPC procedure added to `server/routers.ts` or any `server/routers/*.ts` file must be accompanied by a corresponding unit test that covers at minimum: the happy path, one validation failure (invalid input), and one authorisation failure (unauthenticated or insufficient role). Utility functions in `server/db.ts` must be tested with mocked Drizzle query builders. Business logic functions (e.g., amortisation schedule calculation in the EMI router, FX rate interpolation in the FX router) must be tested with boundary values and edge cases.

5.3 Running Unit Tests

```
# Run all unit tests
PG_DATABASE_URL="postgresql://paygate:paygate_dev_2026@127.0.0.1:5432/paygate_
pnpm test

# Run a specific test file
pnpm test server/nip.settlement.test.ts

# Run with coverage report
pnpm test --coverage
```

6. Layer 2 — Integration Tests (tRPC + PostgreSQL)

6.1 Scope

Integration tests validate the full request path from tRPC procedure invocation through the database layer and back, using a real PostgreSQL instance with seeded data. These tests do not mock the database but do mock external HTTP calls (Go bridge, Python services, NIBSS, Stripe).

6.2 Critical Integration Test Scenarios

The following scenarios represent the highest-risk integration paths and must be covered by dedicated test cases:

Payment Processing

A transaction initiated via `trpc.transactions.create` must persist a row in the `transactions` table with the correct `status`, `amount_kobo`, `currency`, `merchant_id`, and `idempotency_key`. A duplicate request with the same idempotency key must return the original transaction without creating a new row. A transaction exceeding the merchant's daily limit must be rejected with a `TRANSACTION_LIMIT_EXCEEDED` error code.

Payout Approval Workflow

A payout request created via `trpc.payouts.request` must enter `pending_approval` status. Approval by a user with the `finance_approver` role must transition the status to `approved` and trigger the NIBSS NIP transfer. Rejection must transition to `rejected` with a mandatory reason field. Attempting approval by a user without the `finance_approver` role must return a `FORBIDDEN` tRPC error.

KYB Verification Lifecycle

A merchant KYB submission must create a `kyb_verifications` record and associated `kyb_steps` records for each verification step (CAC check, director ID, bank account). Each step must be independently updatable. Full approval must transition the merchant's `kyb_status` to `approved` and unlock payout capabilities.

Webhook Delivery and Retry

A webhook event created via the internal `webhookRetry` worker must be delivered to the configured endpoint. On HTTP 5xx response, the worker must retry with exponential backoff (1s, 2s, 4s, 8s, 16s). After 5 failed attempts, the delivery must be marked `failed` and a notification sent to the merchant's registered email.

6.3 Database Integrity Tests

All 167 tables must be validated for referential integrity after seed data insertion. The `smoke.test.ts` file covers this systematically. Additionally, the following constraints must be explicitly tested:

Constraint	Table	Test
Unique idempotency keys	transactions	Insert duplicate key → expect unique violation
Non-negative balances	wallets	Update balance to -1 → expect check constraint violation
Valid enum values	transactions.status	Insert invalid status → expect enum violation
Cascade deletes	merchants → api_keys	Delete merchant → verify api_keys deleted
Tenant isolation	All tenant-scoped tables	Query with wrong tenant_id → expect empty result

7. Layer 3 — Service Contract Tests (Go Bridge + Python Services)

7.1 Go Bridge Contract Tests

The Go bridge exposes 257 HTTP routes across 29 handler files. Each route must be tested for: correct HTTP method acceptance, request body validation (JSON schema), response structure conformance, and error response format. The following handler groups are highest priority:

Handler Group	Routes	Contract Requirements
<code>payments_handler.go</code>	POST /v1/payments/initiate, GET /v1/payments/{id}	Amount in kobo (integer), currency ISO 4217, idempotency header required
<code>nip_handler.go</code>	POST /v1/nip/transfer, GET /v1/nip/banks	Bank code 6 digits, account number 10 digits, amount > 0
<code>compliance_handler.go</code>	POST /v1/kyc/verify, GET /v1/aml/screen	BVN 11 digits, NIN 11 digits, response includes risk_score 0–100
<code>fx_handler.go</code>	GET /v1/fx/rates, POST /v1/fx/convert	Base/quote currency pair, rate with 6 decimal precision
<code>lending_handler.go</code>	POST /v1/loans/apply, GET /v1/loans/{id}/schedule	Principal in kobo, tenure in months, amortisation schedule array
<code>usdc_handler.go</code>	POST /v1/usdc/payout, GET /v1/usdc/balance	Wallet address EIP-55 checksum, amount in USDC with 6 decimals

Contract tests must be written using Go's `net/http/httptest` package and run as part of `go test ./...` in the `go-bridge/` directory.

7.2 Python Service Contract Tests

Each of the 30 Python services must have a `test_main.py` file in its directory that validates the Flask application routes using `pytest` and `flask.testing.FlaskClient`. The minimum test coverage per service is:

Service	Endpoint to Test	Validation
fraud-scoring	POST /score	Returns <code>risk_score</code> (0.0–1.0), <code>decision</code> (approve/review/decline)
emi-service	POST /calculate	Returns amortisation schedule with correct monthly instalments
intl-remittance	POST /transfer	Validates corridor (NG→GH, NG→KE, NG→ZA), FX rate applied
mutual-funds	GET /nav/{fund_id}	Returns NAV with date, price per unit, AUM
pension-nps	POST /contribution	Validates RSA PIN format, employer/employee split
wealth-management	POST /goals	Returns projected value with compound growth calculation
cashback-rewards	POST /earn	Returns cashback amount based on tier rules
bulk-collections	POST /batch	Validates batch size ≤ 1000 , returns <code>batch_id</code>
salary-accounts	POST /payroll	Validates payroll date, employee count, total disbursement
soundbox	POST /event	Returns audio notification payload for POS soundbox device

7.3 Rust Service Contract Tests

Each Rust service must pass `cargo test` in its directory. The following specific behaviours must be validated:

Service	Test Scenario
billing-engine	Invoice generation with correct tax calculation (7.5% VAT for Nigeria)
credit-scoring	Score output in range 300–850, deterministic for same input features
inventory-engine	Stock decrement on sale, reorder trigger at threshold
loyalty-ledger	Points accrual on transaction, tier upgrade at threshold
tigerbeetle-recon	Double-entry bookkeeping: debits always equal credits
wallet-ffi	FFI boundary safety: no memory leaks on repeated calls

8. Layer 4 — End-to-End UI Tests (Playwright)

8.1 Spec Files and Coverage

Three Playwright spec files cover the full merchant portal UI:

Spec File	Test Count	Coverage
e2e/merchant-portal.spec.ts	20+	All 20 primary merchant pages: navigation, data rendering, CRUD operations, search, filters
e2e/business-workflows.spec.ts	15+	Onboarding wizard, KYB submission, payout approval, dispute workflow, team management
e2e/api-integration.spec.ts	15+	API health endpoints, tRPC procedure calls via browser, webhook configuration, API key generation

8.2 Critical E2E Test Flows

The following flows represent the highest business value and must never be broken:

Merchant Onboarding Flow

1. Navigate to `/onboarding` as an unauthenticated user.
2. Complete the business information form (business name, RC number, business type, address).
3. Upload CAC certificate and director ID documents.
4. Submit KYB verification request.
5. Assert that the merchant dashboard shows `KYB Pending` status.
6. Simulate admin approval via the admin panel.
7. Assert that the merchant dashboard shows `KYB Approved` and the payout button is enabled.

Transaction Processing Flow

1. Log in as a verified merchant.
2. Navigate to `/transactions`.
3. Use the search bar to filter by date range and status.
4. Assert that the transaction table renders with correct columns (Reference, Amount, Status, Customer, Date).
5. Click a transaction row to open the detail drawer.
6. Assert that the detail drawer shows the full transaction lifecycle (initiated → processing → completed).
7. Test the CSV export button and assert that the downloaded file contains the filtered rows.

Payout Approval Flow

1. Log in as a merchant with the `finance_approver` role.
2. Navigate to `/payouts`.
3. Create a new payout request for ₦500,000.
4. Assert that the payout enters `pending_approval` status.
5. Log in as a second approver (dual-control requirement).
6. Approve the payout.
7. Assert that the payout transitions to `approved` and a success toast is shown.
8. Assert that the payout appears in the settlement history.

Virtual Card Issuance Flow

1. Navigate to `/virtual-cards`.
2. Click “Issue New Card”.
3. Fill in cardholder name, spending limit, and currency (USD).
4. Assert that the new card appears in the card list with masked PAN.
5. Click “Freeze Card” and assert status changes to `frozen`.
6. Click “Unfreeze Card” and assert status returns to `active`.

Dispute Resolution Flow

1. Navigate to `/disputes`.
2. Open an existing dispute in `open` status.
3. Upload supporting evidence (PDF).
4. Submit the dispute response.
5. Assert that the dispute transitions to `under_review`.
6. Simulate chargeback resolution via the admin panel.
7. Assert that the dispute closes with the correct outcome (won/lost).

FX Dashboard Flow

1. Navigate to `/fx`.
2. Assert that live FX rates are displayed for USD/NGN, GBP/NGN, EUR/NGN.
3. Enter a conversion amount of \$1,000 USD.
4. Assert that the NGN equivalent is calculated and displayed.
5. Initiate a cross-border transfer.
6. Assert that the transfer appears in the cross-border history table.

8.3 Responsive Design Tests

All 96 pages must be tested at three viewport sizes: mobile (375×812), tablet (768×1024), and desktop (1440×900). The Playwright config must include these viewports in the `projects` array. Critical assertions for responsive tests include: navigation sidebar collapses to hamburger menu on mobile, tables scroll horizontally

on mobile, modals do not overflow viewport, and form inputs are accessible on touch devices.

8.4 Running Playwright Tests

```
# Install Playwright browsers (first time only)
pnpm exec playwright install chromium

# Run all E2E tests against local dev server
PLAYWRIGHT_BASE_URL=http://localhost:3000 pnpm exec playwright test

# Run against staging
PLAYWRIGHT_BASE_URL=https://staging.paygate.ng pnpm exec playwright test

# Run with UI mode for debugging
pnpm exec playwright test --ui

# Generate HTML report
pnpm exec playwright test --reporter=html
```

9. Layer 5 — Smoke Tests (Production Readiness)

9.1 Scope

The smoke test suite (`server/smoke.test.ts`) validates that all 167 database tables are accessible, all critical tRPC procedures respond, and all infrastructure dependencies are reachable. These tests run in under 60 seconds and are the first gate in the CI pipeline.

9.2 Smoke Test Checklist

The following checks must pass before any deployment proceeds:

Check	Expected Result
PostgreSQL connection	<code>SELECT 1</code> returns 1 within 500ms
All 167 tables accessible	<code>SELECT COUNT(*)</code> on each table returns without error
<code>trpc.auth.me</code> procedure	Returns user object or unauthenticated error (not 500)
<code>trpc.transactions.list</code> procedure	Returns paginated array (may be empty)
<code>trpc.merchants.getProfile</code> procedure	Returns merchant object for seeded test merchant
Go bridge health endpoint	<code>GET /health</code> returns <code>{"status": "ok"}</code> within 1s
Redis connectivity	<code>PING</code> returns <code>PONG</code> within 200ms
Temporal worker connectivity	Workflow service responds within 2s
Keycloak OIDC discovery	<code>/.well-known/openid-configuration</code> returns valid JSON
OTEL Collector	Traces endpoint accepts a test span without error

9.3 Running Smoke Tests

```
PG_DATABASE_URL="postgresql://paygate:paygate_dev_2026@127.0.0.1:5432/paygate_
\
pnpm test server/smoke.test.ts
```

10. Layer 6 — Performance and Load Tests

10.1 Tools and Approach

Performance tests use [k6](#) for HTTP load testing and [pgbench](#) for database throughput. Tests are run against the staging environment only, never against production. All

performance tests must be preceded by a database vacuum and statistics reset to ensure consistent baselines.

10.2 Performance Targets

Endpoint	P50 Latency	P95 Latency	P99 Latency	Max TPS
POST <code>/api/trpc/transactions.create</code>	< 80ms	< 200ms	< 500ms	500 TPS
GET <code>/api/trpc/transactions.list</code>	< 50ms	< 150ms	< 300ms	1,000 TPS
POST <code>/api/trpc/payouts.request</code>	< 100ms	< 300ms	< 800ms	100 TPS
GET <code>/api/trpc/analytics.dashboard</code>	< 200ms	< 500ms	< 1,000ms	200 TPS
POST <code>/v1/nip/transfer</code> (Go bridge)	< 150ms	< 400ms	< 1,000ms	200 TPS
Python fraud scoring	< 200ms	< 600ms	< 1,500ms	100 TPS

10.3 Load Test Scenarios

Scenario 1 — Baseline Load: 50 virtual users, 5-minute ramp-up, 10-minute sustained load, 2-minute ramp-down. Validates that the system handles normal business-hours traffic without degradation.

Scenario 2 — Peak Load (End of Month): 500 virtual users, simulating the Nigerian end-of-month salary payment surge. All users execute the payout request flow simultaneously. The system must maintain P95 latency under 800ms and zero error rate.

Scenario 3 — Database Connection Pool Exhaustion: Gradually increase concurrent connections beyond the pool limit (default: 20). The system must queue requests and serve them in order rather than returning connection errors. No requests should fail with a 500 error; they may return 429 (Too Many Requests) after a configurable timeout.

Scenario 4 — Large Batch Processing: Submit a bulk collection batch of 1,000 records via `POST /v1/bulk/collections`. The system must process all records within 30 seconds and return a batch status endpoint that can be polled for progress.

10.4 k6 Test Script Location

```
tests/  
  load/  
    baseline.js  
    peak-load.js  
    connection-pool.js  
    bulk-batch.js  
    k6-config.json
```

11. Layer 7 — Security Tests

11.1 Automated Security Scanning

The following automated security tools must be integrated into the CI pipeline:

Tool	Purpose	Frequency
<code>npm audit / pnpm audit</code>	Node.js dependency vulnerability scan	Every push
<code>cargo audit</code>	Rust dependency vulnerability scan	Every push
<code>bandit</code>	Python static security analysis	Every push
<code>gosec</code>	Go static security analysis	Every push
<code>trivy</code>	Docker image vulnerability scan	Every build
OWASP ZAP (baseline)	Dynamic application security testing	Nightly against staging

11.2 Manual Security Test Scenarios

The following scenarios must be executed manually before every major release:

Authentication and Authorisation

Every protected tRPC procedure must return a `401 UNAUTHORIZED` error when called without a valid session cookie. Every admin-only procedure must return a `403 FORBIDDEN` error when called by a user with the `user` role. Session cookies must have the `HttpOnly`, `Secure`, and `SameSite=Strict` attributes in production. JWT tokens must expire after the configured TTL (default: 24 hours) and must not be accepted after expiry.

Input Validation and Injection Prevention

All tRPC procedure inputs are validated by Zod schemas. The following injection payloads must be tested against every string input field: SQL injection (`' OR 1=1 --`), NoSQL injection (`{"$gt": ""}`), XSS (`<script>alert(1)</script>`), and path traversal (`../../etc/passwd`). All payloads must be sanitised and return a `400 BAD_REQUEST` error with a descriptive validation message.

Rate Limiting

The API must enforce rate limiting at the following thresholds: 100 requests per minute per IP for unauthenticated endpoints, 1,000 requests per minute per authenticated user for standard procedures, and 10 requests per minute per user for sensitive operations (password reset, API key generation, payout initiation). Exceeding these limits must return `429 TOO_MANY_REQUESTS` with a `Retry-After` header.

Webhook Signature Verification

All incoming webhooks (Stripe, NIBSS, M-Pesa) must be verified using HMAC-SHA256 signature validation. A webhook with an invalid or missing signature must be rejected with `401 UNAUTHORIZED`. The raw request body must be used for signature verification (not the parsed JSON body).

PCI DSS Compliance Checks

The platform must never log, store, or transmit full card numbers, CVV codes, or card expiry dates. All card data must be tokenised via Stripe before touching the PayGate backend. The `server/security.test.ts` file validates these constraints programmatically.

11.3 OWASP Top 10 Coverage Matrix

OWASP Risk	Mitigation in PayGate	Test Coverage
A01 Broken Access Control	<code>protectedProcedure</code> , <code>adminProcedure</code> , role checks	<code>security.test.ts</code> , <code>validation.test.ts</code>
A02 Cryptographic Failures	HTTPS only, JWT signing, bcrypt passwords	<code>production.hardening.test.ts</code>
A03 Injection	Zod validation, Drizzle parameterised queries	<code>validation.test.ts</code> , <code>security.test.ts</code>
A04 Insecure Design	Dual-control payout approval, KYB gating	<code>disputes.test.ts</code> , <code>wave77.production.test.ts</code>
A05 Security Misconfiguration	Helmet.js headers, CORS whitelist	<code>security.test.ts</code>
A06 Vulnerable Components	<code>pnpm audit</code> , <code>cargo audit</code> , <code>bandit</code>	CI pipeline
A07 Auth Failures	Keycloak OIDC, session expiry, PKCE	<code>keycloak.auth.test.ts</code> , <code>auth.logout.test.ts</code>
A08 Software Integrity	Docker image signing, dependency lockfiles	CI pipeline
A09 Logging Failures	Structured JSON logs, OTEL traces	<code>smoke.test.ts</code>
A10 SSRF	Allowlist for outbound HTTP calls	<code>security.test.ts</code>

12. Layer 8 — Chaos and Resilience Tests

12.1 Chaos Engineering Principles

Chaos tests validate that the system degrades gracefully when individual components fail. All chaos tests must be run against a dedicated chaos environment (not staging or

production) and must be preceded by a full system health check. Each chaos scenario must define a steady-state hypothesis, the chaos injection method, and the expected system behaviour.

12.2 Chaos Scenarios

Scenario C1 — PostgreSQL Primary Failure

Hypothesis: The portal continues to serve read-only cached data when the primary PostgreSQL instance is unavailable.

Injection: `docker stop postgres-primary`

Expected: Write operations return `503 SERVICE_UNAVAILABLE` with a user-friendly error message. Read operations served from Redis cache for up to 60 seconds. Automatic reconnection within 30 seconds of PostgreSQL recovery.

Scenario C2 — Go Bridge Unavailability

Hypothesis: tRPC procedures that depend on the Go bridge return graceful fallback responses rather than 500 errors.

Injection: `docker stop go-bridge`

Expected: Procedures calling `callBridge()` return a `SERVICE_UNAVAILABLE` tRPC error with `retryable: true`. The portal UI shows a “Payment services temporarily unavailable” banner. No unhandled promise rejections in the browser console.

Scenario C3 — Redis Cache Failure

Hypothesis: The system falls back to direct database queries when Redis is unavailable, with acceptable latency degradation.

Injection: `docker stop redis`

Expected: P95 latency increases by no more than 3× baseline. No errors returned to users. Redis reconnection attempted every 5 seconds.

Scenario C4 — Python Fraud Scoring Service Failure

Hypothesis: Transactions are not blocked when the fraud scoring service is unavailable; they are flagged for manual review instead.

Injection: `docker stop fraud-scoring`

Expected: Transactions proceed with `fraud_review_required: true` flag. A Grafana alert fires within 60 seconds of the service going down. The fraud scoring service restarts automatically via Docker health check.

Scenario C5 — Network Partition (Go Bridge ↔ NIBSS)

Hypothesis: NIP transfer requests are queued and retried when the NIBSS gateway is unreachable.

Injection: `iptables -A OUTPUT -d nibss-gateway.ng -j DROP`

Expected: Transfer requests enter `pending` status. The retry worker attempts redelivery with exponential backoff. After 5 failed attempts, the transfer is marked `failed` and the merchant is notified. No duplicate charges occur due to idempotency key enforcement.

Scenario C6 — Memory Pressure on Portal Server

Hypothesis: The Node.js portal server does not crash under memory pressure; it sheds load gracefully.

Injection: `stress-ng --vm 1 --vm-bytes 80%` on the portal container

Expected: The server activates the circuit breaker and returns `503` for new requests. Existing in-flight requests complete. Memory usage returns to baseline within 60 seconds of stress removal.

Scenario C7 — Temporal Worker Crash

Hypothesis: Long-running workflows (KYB, payout approval) resume from their last checkpoint after a Temporal worker crash.

Injection: `docker kill temporal-worker`

Expected: Active workflows are paused. After worker restart, workflows resume from the last persisted checkpoint. No workflow state is lost. Workflow history is intact in the Temporal UI.

Scenario C8 — Clock Skew

Hypothesis: JWT tokens and idempotency keys remain valid under moderate clock skew between services.

Injection: Set system clock on the portal server 5 minutes ahead.

Expected: JWT validation allows up to 5 minutes of clock skew (configurable via `clockTolerance`). Idempotency keys with timestamp-based TTLs are not prematurely expired. OTEL trace timestamps are corrected by the collector.

13. Business Workflow Test Scenarios

13.1 Core Payment Lifecycle

The following end-to-end business workflow must be tested as a single atomic scenario, covering all system layers from UI to database:

NIP Transfer Lifecycle (₦50,000 from merchant wallet to customer)

1. Merchant initiates transfer via `trpc.transactions.create` with `type: "nip_transfer"`.
2. Go bridge `POST /v1/nip/transfer` is called with the transfer details.
3. NIBSS gateway (mocked in test) returns session ID.
4. Transaction status transitions: `initiated` → `processing` → `completed`.
5. Merchant wallet balance decreases by ₦50,000 + fees.
6. Customer receives credit notification via push service.
7. Settlement record created in `settlements` table.
8. Webhook delivered to merchant's configured endpoint.
9. Transaction appears in merchant dashboard with correct status.
10. Analytics aggregation updated (daily transaction volume).

13.2 Lending Workflow

Merchant Loan Disbursement (₦5,000,000 working capital loan)

1. Merchant applies via `trpc.lending.apply` with business financials.
2. Credit scoring Rust service evaluates application (score must be ≥ 650 for approval).
3. Loan offer generated with amortisation schedule (12 monthly instalments).
4. Merchant accepts offer via digital signature.
5. Loan disbursed to merchant's settlement account within 24 hours.
6. First repayment scheduled via Temporal workflow.
7. Monthly repayment deducted automatically from settlement proceeds.

8. Loan status transitions: `applied` → `under_review` → `approved` → `disbursed` → `active` → `closed`.

13.3 BNPL Checkout Flow

Customer BNPL Purchase (₦120,000 split into 3 instalments)

1. Customer selects BNPL at checkout via `trpc.bnpl.initiate`.
2. Credit check performed (BVN-linked credit bureau query).
3. BNPL plan created: ₦40,000 × 3 monthly instalments.
4. First instalment collected immediately.
5. Merchant receives full ₦120,000 (minus BNPL fee) within 24 hours.
6. Subsequent instalments collected automatically on due dates.
7. Late payment triggers SMS/push notification and 1.5% penalty.

13.4 Cross-Border Remittance Flow

Nigeria → Ghana Remittance (NGN 500,000 → GHS equivalent)

1. Sender initiates via `trpc.crossborder.initiate` with recipient GHS account.
2. FX rate fetched from `fx-rate-feed` Python service (NGN/GHS rate).
3. Compliance check: AML screening, transaction limit validation.
4. Transfer routed via M-Pesa connector (for GHS mobile money) or SWIFT.
5. Recipient notified via SMS.
6. Transfer status: `initiated` → `fx_locked` → `processing` → `completed`.
7. Both sender and recipient receive transaction receipts.

13.5 Subscription Billing Lifecycle

SaaS Merchant Subscription (₦50,000/month plan)

1. Merchant selects plan via `trpc.subscriptions.subscribe`.
2. Stripe checkout session created and merchant redirected.
3. Stripe webhook `checkout.session.completed` received and verified.

4. Subscription record created in `subscription_plans_v2` table.
 5. Monthly invoice generated automatically.
 6. Failed payment triggers dunning workflow (retry on day 3, 7, 14).
 7. Subscription cancelled after 3 failed payment attempts.
 8. Merchant notified at each dunning stage.
-

14. Data Integrity and Compliance Tests

14.1 Tenant Isolation

Every database query in the system must be scoped to the authenticated merchant's `tenant_id`. The following test must be executed for every tRPC procedure that returns data: create two merchants (Merchant A and Merchant B) with separate tenant IDs, seed data for both, authenticate as Merchant A, and assert that no data belonging to Merchant B is returned in any query response.

14.2 Audit Trail Completeness

Every financial operation (transaction, payout, loan disbursement, refund, chargeback) must create an entry in the `audit_logs` table with the following fields: `actor_id`, `action`, `resource_type`, `resource_id`, `before_state` (JSON), `after_state` (JSON), `ip_address`, `user_agent`, and `timestamp`. The audit log must be immutable (no UPDATE or DELETE allowed on `audit_logs` rows).

14.3 CBN Regulatory Compliance

The following CBN (Central Bank of Nigeria) regulatory requirements must be validated by automated tests:

Requirement	Validation
Daily transaction limit per customer: ₦5,000,000	<code>validation.test.ts</code> — reject transactions exceeding limit
BVN verification for transactions > ₦50,000	<code>compliance.test.ts</code> — require BVN for high-value transactions
AML screening for transactions > ₦1,000,000	<code>aml.test.ts</code> — trigger AML check, block on positive match
KYC tier limits (Tier 1: ₦300,000/day, Tier 2: ₦500,000/day, Tier 3: ₦5,000,000/day)	<code>tier1to5.test.ts</code> , <code>tier6to8.test.ts</code>
Transaction reporting to NFIU for > ₦5,000,000	<code>compliance.test.ts</code> — verify NFIU report generated
POS terminal registration with CBN	<code>smoke.test.ts</code> — verify <code>terminal_id</code> format

14.4 GDPR / NDPR Data Privacy

The platform must comply with Nigeria’s NDPR (Nigeria Data Protection Regulation). The following tests validate data privacy controls:

1. Customer data deletion request via `trpc.privacy.deleteAccount` must anonymise all PII fields (name, email, phone, BVN) while retaining transaction records for regulatory purposes.
 2. Data export request via `trpc.privacy.exportData` must return a complete JSON export of all customer data within 72 hours.
 3. Consent records must be stored in `privacy_settings` table with timestamp and version.
 4. Third-party data sharing (credit bureaus, AML providers) must be logged in the audit trail.
-

15. Observability and Monitoring Validation

15.1 OpenTelemetry Trace Validation

After every significant transaction, a trace must be visible in Grafana Tempo with the following spans:

- `portal.trpc.procedure` — the tRPC procedure invocation
- `portal.db.query` — each PostgreSQL query executed
- `portal.bridge.http` — the Go bridge HTTP call (if applicable)
- `go-bridge.handler` — the Go handler processing
- `python.service.request` — the Python service call (if applicable)

The trace must show the full request path from browser to database and back, with no gaps. P99 trace latency must not exceed 2 seconds for any transaction flow.

15.2 Prometheus Metrics Validation

The following metrics must be scraped by Prometheus and visible in Grafana dashboards:

Metric	Type	Alert Threshold
<code>paygate_transactions_total</code>	Counter	—
<code>paygate_transaction_amount_kobo</code>	Histogram	—
<code>paygate_nip_transfer_duration_ms</code>	Histogram	P95 > 2,000ms → alert
<code>paygate_fraud_score_p95</code>	Gauge	> 0.8 → alert
<code>paygate_payout_approval_pending</code>	Gauge	> 50 → alert
<code>paygate_webhook_delivery_failures</code>	Counter	> 10/min → alert
<code>paygate_db_connection_pool_used</code>	Gauge	> 80% → alert
<code>paygate_go_bridge_error_rate</code>	Gauge	> 1% → alert

15.3 Log Aggregation Validation

All services must ship structured JSON logs to Loki via Promtail. Each log entry must include: `service`, `level` (info/warn/error), `trace_id`, `span_id`, `merchant_id` (where applicable), and `message`. Error logs must include the full stack trace. Log retention must be configured for 90 days in production.

16. CI/CD Pipeline Integration

16.1 Recommended Pipeline Structure

```
# .github/workflows/ci.yml (abbreviated)
jobs:
  unit-integration:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_PASSWORD: paygate_ci
          POSTGRES_DB: paygate_test
    steps:
      - uses: actions/checkout@v4
      - run: pnpm install
      - run: pnpm db:push
      - run: node seed-bootstrap.mjs && node seed-full.mjs
      - run: pnpm test
      - uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage/

  e2e:
    needs: unit-integration
    runs-on: ubuntu-latest
    steps:
      - run: pnpm exec playwright install --with-deps chromium
      - run: pnpm dev &
      - run: pnpm exec playwright test --reporter=github

  security:
    runs-on: ubuntu-latest
    steps:
      - run: pnpm audit --audit-level=high
      - run: cd go-bridge && gosec ./...
      - run: cd python-services && bandit -r . -ll
      - run: trivy image paygate-portal:latest
```

16.2 Quality Gates

No deployment may proceed unless all of the following gates pass:

Gate	Threshold
Unit/integration test pass rate	100% (zero failures)
TypeScript compilation	Zero errors
Code coverage (server/)	$\geq 80\%$ line coverage
Smoke tests	100% pass
Security audit	Zero high/critical vulnerabilities
E2E tests (critical flows only)	100% pass
Performance regression	P95 latency within 20% of baseline

17. Test Data Management

17.1 Seed Data Strategy

The project includes the following seed scripts, which must be run in order:

Script	Purpose	Tables Covered
<code>seed-bootstrap.mjs</code>	Foundational data: tenants, users, merchants	3 tables
<code>seed-full.mjs</code>	Core business data: transactions, customers, API keys	20+ tables
<code>seed-wave85-complete.mjs</code>	Advanced features: BNPL, virtual cards, lending	30+ tables
<code>seed-all-tables.mjs</code>	Remaining tables: consumer features, compliance, analytics	91 tables
<code>seed-fix-final.mjs</code>	Final FK fixes: cross-border, consumer wallets	3 tables
<code>seed-consumer-final.mjs</code>	Consumer-side tables: P2P, split bill, red envelopes	11 tables

17.2 Test Data Isolation

Each test suite must use a dedicated test tenant ID to prevent data contamination between test runs. The recommended pattern is to create a test tenant at the start of each test file and delete it (with cascade) at the end. For integration tests that require seeded data, use the `beforeAll` hook to insert test-specific records and the `afterAll` hook to clean up.

17.3 Sensitive Data Handling

Test data must never include real BVN numbers, real bank account numbers, real card numbers, or real personal information. All test data must use the following synthetic formats:

- BVN: `22222222222` (11 twos — reserved for testing)
 - Bank account: `00000000000` to `00000000099`
 - Card number: `4242424242424242` (Stripe test card)
 - Phone: `+23480000000000` to `+23480000000099`
-

18. Defect Classification and SLAs

18.1 Severity Levels

Severity	Definition	Resolution SLA
P0 — Critical	Production down, data loss, security breach, financial discrepancy	2 hours
P1 — High	Core payment flow broken, authentication failure, data corruption	8 hours
P2 — Medium	Feature degraded but workaround available, UI broken for some users	48 hours
P3 — Low	Minor UI issue, non-critical feature broken, cosmetic defect	1 sprint
P4 — Enhancement	Feature request, performance improvement, refactoring	Backlog

18.2 Defect Reporting Requirements

Every defect report must include: severity classification, steps to reproduce (numbered, specific), expected behaviour, actual behaviour, environment (local/staging/production), browser/OS version, screenshot or screen recording, trace ID from Grafana Tempo (for production issues), and the specific test case that should have caught this defect.

19. Test Coverage Targets

19.1 Coverage by Layer

Layer	Current State	Target	Measurement
Unit/Integration (Vitest)	2,016 tests, 58 files	2,500+ tests	<code>pnpm test -- coverage</code>
E2E (Playwright)	50+ test cases, 3 spec files	100+ test cases	Playwright HTML report
Smoke	58 checks	80 checks (all 167 tables + infra)	Vitest pass rate
Performance	0 automated	4 k6 scenarios	k6 HTML report
Security	Manual + <code>security.test.ts</code>	OWASP ZAP baseline + manual	ZAP HTML report
Chaos	0 automated	8 scenarios	Chaos runbook

19.2 Coverage by Service

Service	Unit Tests	Integration Tests	Contract Tests
Portal (tRPC)	✅ Comprehensive	✅ Comprehensive	N/A
Go Bridge	❌ Missing	❌ Missing	⚠️ Partial
Python Services (30)	❌ Missing	❌ Missing	❌ Missing
Rust Services (6)	⚠️ Partial (cargo test)	❌ Missing	❌ Missing
Database (167 tables)	N/A	✅ Smoke coverage	N/A
E2E UI (96 pages)	N/A	N/A	⚠️ 20 pages covered

The most significant gap is the absence of Go bridge unit tests and Python service pytest suites. These must be prioritised in the next engineering sprint.

20. Appendix — Test File Index

20.1 Existing Vitest Test Files (58 files)

The following test files exist in `server/` and are run by `pnpm test`:

`auth.logout.test.ts` · `comprehensive.test.ts` · `crossborder.wave4.test.ts` ·
`db.pg.test.ts` · `disputes.test.ts` · `keycloak.auth.test.ts` · `new-`
`features.test.ts` · `nip.retry.test.ts` · `nip.settlement.test.ts` ·
`paygate.integration.test.ts` · `production.hardening.test.ts` ·
`security.test.ts` · `slaEscalation.test.ts` · `smoke.test.ts` ·
`tier1to5.test.ts` · `tier6to8.test.ts` · `usdc.router.test.ts` ·
`validation.test.ts` · `wallet.crossborder.test.ts` · `wave5.test.ts` through
`wave80.test.ts` (47 wave files covering incremental feature additions)

20.2 Playwright E2E Spec Files (3 files)

`e2e/merchant-portal.spec.ts` — All 20 primary merchant pages
`e2e/business-workflows.spec.ts` — Onboarding, KYB, payout approval, disputes
`e2e/api-integration.spec.ts` — API health, tRPC procedures, webhook configuration

20.3 Tests to Be Created (Priority Order)

Priority	Test File	Description
1	go-bridge/tests/payments_test.go	Go bridge payment handler unit tests
2	go-bridge/tests/nip_test.go	NIP transfer handler unit tests
3	python-services/fraud-scoring/test_main.py	Fraud scoring API contract tests
4	python-services/emi-service/test_main.py	EMI calculation contract tests
5	python-services/intl-remittance/test_main.py	Remittance corridor contract tests
6	tests/load/baseline.js	k6 baseline load test
7	tests/load/peak-load.js	k6 end-of-month peak load test
8	e2e/consumer-portal.spec.ts	Consumer portal E2E tests (24 pages)
9	e2e/admin-portal.spec.ts	Admin portal E2E tests
10	tests/chaos/postgres-failure.sh	Chaos: PostgreSQL primary failure

This document is a living artefact and must be updated whenever new features, services, or compliance requirements are added to the PayGate platform. The testing plan owner is the Engineering Lead, and all changes must be reviewed and approved before merging to the `main` branch.